

SyncQuest

Technical Deep Dive

Every mechanism, file, and protocol explained -- Distributed Systems Project, Part 1

This document is the companion to the project report and the cross-device guide. Where those explain *what* the app does and *how to run it*, this one explains *how it actually works internally* -- file by file, message by message -- including two real races/bugs found and fixed during testing, not just the intended design.

1. Project Layout

Two independent Node.js projects live under one repo root. They do not import each other's code and communicate only over the network (WebSocket), which is the point: the signaling server can be redeployed, restarted, or replaced without touching the frontend's data model.

```
p2p-editor/
  signaling-server/
    server.js      # the entire backend, well under 200 lines
    package.json   # one dependency: ws
    Dockerfile
  frontend/
    src/
      hooks/
        useYjsRoom.js  # owns Y.Doc, WebRTCProvider, IndexeddbPersistence
        useToasts.js   # cosmetic notification feed (peer diff + achievement)
      components/
        Editor.jsx      # Quill + y-quill binding
        PresencePanel.jsx # renders peers[] from useYjsRoom
        ConnectionStatus.jsx
        RoomBar.jsx
        QuestProgress.jsx # cosmetic XP bar from live Y.Text length
        Toasts.jsx
      utils/randomIdentity.js
      App.jsx           # room-id/identity bootstrap, wires hooks to components
      main.jsx
      vite.config.js    # host:true so LAN devices can reach the dev server
      package.json      # root convenience scripts (npm run signaling / dev)
```

2. The Signaling Server -- Wire Protocol

The entire server-side state is one JavaScript object: `Map<roomName, Set<WebSocket>>`, called `topics` in the code. There is no database, no session store, nothing written to disk. It implements four message types, all JSON over a plain WebSocket connection:

subscribe	Client asks to join one or more room names: <code>{type: 'subscribe', topics: [roomId]}</code> . Server adds the socket to <code>topics.get(roomId)</code> .
publish	Client sends an opaque payload to everyone else subscribed to a room: <code>{type: 'publish', topic: roomId, ...offerOrAnswerOrIce}</code> . Server fans it out to every other socket in that room's Set -- it never inspects the payload beyond reading topic.
unsubscribe	Removes the socket from a room's Set.
ping / pong	Client-initiated keepalive; server replies <code>{type: 'pong'}</code> . The server also runs its own WebSocket-level ping every 30s and closes any socket that doesn't answer, to garbage-collect dead connections and keep topics accurate.

In practice, the payload of a `publish` message is a WebRTC SDP offer, an SDP answer, or an ICE candidate -- generated and consumed entirely by the browser's own `RTCPeerConnection` API (via the `simple-peer` library that `y-webrtc` wraps). The server code has no branch that parses SDP, no `Yjs` import, and no reference to `Y.Doc` anywhere -- there is no code path by which document content could reach this process, not just a policy against it.

```
// signaling-server/server.js -- the entire routing logic for 'publish'
case 'publish': {
  if (!message.topic) break
  const receivers = topics.get(message.topic)
  if (receivers) {
    message.clients = receivers.size
    receivers.forEach((receiver) => send(receiver, message))
  }
  break
}
```

3. useYjsRoom.js -- The Sync Stack

This hook is the single place that wires together the three libraries doing the actual distributed-systems work: yjs (the CRDT), y-webrtc (the transport/discovery), and y-indexeddb (local persistence). Everything else in the app just reads state this hook derives.

```
frontend/src/hooks/useYjsRoom.js
```

3.1 Object construction (lazy useState, not useEffect)

```
const [doc] = useState(() => new Y.Doc())
const [persistence] = useState(() => new IndexeddbPersistence(`p2p-editor-${roomId}`, doc))
const [provider] = useState(() => new WebRTCProvider(roomId, doc, {
  signaling: [SIGNALING_URL],
  peerOpts: { config: { iceServers: [{ urls: 'stun:stun.l.google.com:19302' }] } }
}))
```

These use `useState(() => ...)` lazy initializers rather than creating the objects in a `useEffect`. That matters for two reasons: (1) it makes `doc` and `provider` stable, render-visible values on the very first render, instead of only existing after an effect runs (which would need an extra state update just to expose them); and (2) it guarantees exactly one `Y.Doc/WebRTCProvider` pair is created per mount of this hook, never re-created on every re-render.

The `App.jsx` caller mounts the component using this hook with `key={roomId}`, so a room change fully unmounts and remounts the whole tree -- tearing down the old `Y.Doc/provider` and constructing a fresh one -- rather than trying to mutate a live provider into pointing at a different room.

3.2 The connection-status state machine

y-webrtc exposes two relevant events on the provider: `'status'` (`{connected}` -- is the signaling WebSocket up) and `'peers'` (`{webRTCpeers, bcPeers}` -- who's actually connected). The hook derives a single UI-facing status from both:

connecting	Default state; neither a peer connection nor the 8s timeout has resolved yet.
connected	<code>activePeerCount > 0</code> -- at least one real peer (<code>webRTCpeers</code>) or same-browser peer (<code>bcPeers</code>) is active.
error	8s timeout elapsed, zero active peers, AND the signaling socket itself never connected.
disconnected	Reserved for an explicit teardown state (not currently reached in normal flow; see Section 5 for exact trigger logic).

```
const onPeers = ({ webRTCpeers, bcPeers }) => {
  activePeerCount = webRTCpeers.length + bcPeers.length
  recomputeStatus()
}
```

`webRTCpeers` are real `RTCPeerConnection` links -- what a genuine cross-device connection uses. `bcPeers` are peers in the *same browser*, connected via the `BroadcastChannel` Web API as a same-origin fast path that completely bypasses WebRTC and the signaling server. Both mean the peer is actively syncing, so the status logic treats them identically -- see Section 5.1 for why this distinction mattered during testing.

3.3 Presence/awareness projection

Presence data (name, color, avatar, class) is not stored in the CRDT document at all -- it lives in `provider.awareness`, an instance of `y-protocols/awareness`'s `Awareness` class (bundled inside the `yjs` package, no separate install needed). Awareness state is deliberately *not* CRDT-merged or persisted: each client publishes its own small JSON blob keyed by its numeric `clientId`, and that entry is dropped the moment the client disconnects (or after a timeout on an unclean disconnect). This is the correct data model for presence -- you want the last-write-wins, ephemeral semantics, not permanent conflict-free history, for something like "where is this cursor right now".

```
provider.awareness.setLocalStateField('user', identity) // publish our own presence

const onAwarenessChange = () => {
  const states = provider.awareness.getStates() // Map<clientId, {user: {...}}>
  const list = []
  states.forEach((state, clientId) => {
    if (state && state.user) {
      list.push({ clientId, name: state.user.name, color: state.user.color,
                  avatar: state.user.avatar, className: state.user.className,
                  self: clientId === doc.clientId })
    }
  })
  setPeers(list)
}
```

Every render of `PresencePanel` is just this array turned into list items -- there is no separate join/leave bookkeeping on the presence side; React re-renders whenever `awareness.on('change', ...)` fires.

3.4 Local persistence: y-indexeddb

`IndexeddbPersistence(`p2p-editor-${roomId}`, doc)` subscribes to the document's own update stream and mirrors every change into the browser's IndexedDB under a room-scoped database name. On construction it also does the reverse: reads whatever was previously stored for this room and applies it to the fresh `Y.Doc` before anything else happens -- which is why a refreshed tab shows its last-known content instantly, before any network round trip completes. The hook exposes this moment as `ready`, set true on the persistence layer's one-time 'synced' event; `App.jsx` uses it to avoid mounting the Quill editor before there is anything to bind to.

Important distinction:

This is *per-browser* durability, not distributed storage. Two different browsers each keep their own IndexedDB copy; there is no shared disk. If every peer closes their tab, the document survives on each of their machines individually but there is no single authoritative copy anywhere -- consistent with there being no central document server at all.

4. The WebRTC Handshake, End to End

Tracing one peer connection from cold start to a synced document:

- Browser A's WebrtcProvider opens a WebSocket to the signaling server and sends `{type:'subscribe', topics:[roomId]}`.
- Browser B does the same for the same roomId. The signaling server now has both sockets in `topics.get(roomId)`.
- y-webrtc deterministically decides, from the two peers' IDs, which side initiates the WebRTC offer (avoiding both sides offering simultaneously).
- The initiator creates an `RTCPeerConnection` (via `simple-peer`) and generates an SDP offer, sent as `{type:'publish', topic:roomId, ...offer}` -- relayed by the server to every other subscriber of that room.
- The receiving browser applies the offer, generates an SDP answer, and publishes it back the same way.
- Both sides exchange ICE candidates the same way (more 'publish' messages) as they discover candidate network paths, using the STUN server (`stun.l.google.com:19302`) to learn their own public-facing address/port for NAT traversal.
- Once ICE completes, the browsers have a direct `RTCDataChannel`. From this point, the signaling server is not involved in anything these two peers do together.
- y-webrtc immediately runs the Yjs sync protocol over that channel: each side sends a compact state-vector describing what it already has, then the other replies with only the updates the first side is missing (this is 'sync step 1/2' in y-webrtc's internal protocol) -- so a newly joined peer catches up in one round trip rather than replaying the whole edit history naively.
- From then on, every local edit is broadcast as a small Yjs update message over the same data channel, and awareness changes (cursor position, presence) are broadcast the same way.

4.1 Two transports, one API

y-webrtc actually supports two independent transports for the same logical 'room', both exposed through the identical `'peers'` event:

webrtcPeers	Real <code>RTCDataChannel</code> connections, established via the handshake above. Works across different devices/browsers, requires the signaling server for discovery.
bcPeers	Same-browser tabs talk over the native <code>BroadcastChannel</code> Web API instead -- no <code>RTCPeerConnection</code> , no ICE, no signaling server involved at all. This exists purely as a fast path/optimization for multi-tab testing on one machine.

Gotcha:

This means opening a second tab in the same browser is NOT a valid way to test whether signaling-dependent peer discovery is working -- same-browser tabs will find each other via `BroadcastChannel` even with the signaling server completely dead. A true test of that path needs a genuinely separate browser context: a different device, or at least a different browser application (e.g. Firefox vs. Chrome) on the same machine.

5. Two Real Bugs Found and Fixed During Testing

These were not caught by code review -- they only surfaced when the app was actually opened in multiple browser tabs and driven end to end. Both are documented here because they illustrate real distributed-systems failure modes (races and hidden transport duality), not just app bugs.

5.1 Status showed "error" while data was actively syncing

The first version of the status logic only counted `webrtcPeers`. Two tabs in the same browser sync perfectly well over `bcPeers` (the `BroadcastChannel` path from Section 4.1), so the document was updating correctly while the status badge nonetheless showed "Connection error", because the count it was watching stayed at zero.

Bug found and fixed during testing:

Changed the peer count to `webrtcPeers.length + bcPeers.length` -- either kind of active connection now counts as "connected", while the underlying distinction is preserved in code comments for anyone reasoning about true cross-device links.

5.2 A newly joined peer got permanently stuck on "connecting"

`WebRTCProvider`'s internal `connect()` runs off a resolved-Promise microtask (`this.key.then(key => ...)`), where `key` is the password-derived encryption key, `null` when no password is set). Microtasks flush before React commits and runs `useEffect` callbacks. The practical consequence: when a third tab joined a room that already had two active peers, the very first 'peers' event -- reporting those two already-present peers -- could fire and complete *before* this hook's `useEffect` had even subscribed to the event. That first, only-relevant-at-startup event was silently missed, and since no further 'peers' event was guaranteed to fire afterward, the status stayed on "connecting"/"error" indefinitely despite the peer connections existing and working underneath.

```
// y-webrtc internals (not this project's code) -- the race:
this.key = password ? cryptoUtils.deriveKey(...) : promise.resolve(null)
this.key.then(key => {
  ...
  if (this.shouldConnect) { this.room.connect() } // <-- fires 'peers' here, via a microtask
})
```

Bug found and fixed during testing:

Added a synchronous catch-up read of `provider.room.webrtcConns.size + provider.room.bcConns.size` immediately after subscribing, inside the same effect. This picks up whatever connection state already existed the instant listening began, closing the race without needing to guess at timing or add an artificial delay.

```
provider.on('peers', onPeers)
// catch-up read: the first 'peers' event can fire before this listener attaches
if (provider.room) {
  activePeerCount = provider.room.webrtcConns.size + provider.room.bcConns.size
  recomputeStatus()
}
```

6. Y.Text Internals -- How Merging Actually Works

`doc.getText( 'quill-content' )` returns a `Y.Text`, a sequence CRDT. Internally, Yjs represents the text as a doubly linked list of small "item" structs, not a plain string. Each item roughly holds:

id	{clientID, clock} -- clientID is a random number generated once per Y.Doc instance (i.e. per browser tab); clock is a per-client monotonically increasing counter, incremented for every local operation. This pair is globally unique across all peers.
content	The actual character(s) inserted.
left / right	References to the neighboring items at the moment of insertion -- an anchor in the sequence's structure, not a numeric array index.
deleted	A tombstone flag; deletions mark an item deleted rather than physically removing it, so concurrent operations referencing it can still resolve consistently.

Because the anchor is "insert after this specific item" rather than "insert at index 5", two peers can type at what looks like the same on-screen position at the same time without needing to agree first. When their two insert operations later meet (propagated over the data channel), each peer runs the same deterministic integration algorithm -- a YATA-style conflict resolution rule that breaks ties using the (`clientID`, `clock`) id when two items want the same left/right anchor. Because every peer runs the identical deterministic rule over the identical set of items (once all updates have propagated), every peer converges to the exact same final linked-list order -- this is what "Strong Eventual Consistency" means concretely: not that conflicts never happen, but that their resolution never requires communication or negotiation, only local computation from data every peer already has.

y-quill (`QuillBinding`) sits between this and the Quill editor: it listens to Quill's own change events (Deltas), translates local edits into `Y.Text` insert/delete operations, and in the other direction, observes the `Y.Text` for remote changes and replays them into Quill as Deltas -- while suppressing the echo of changes that originated locally, so typing doesn't double-apply.

7. The Gamified Layer -- Where It Plugs In (and Where It Doesn't)

Worth being explicit about, since the whole app is themed: exactly two small pieces of UI read from the real CRDT/network state to compute cosmetic output, and nothing writes gameplay state back into the document or the network protocol.

7.1 QuestProgress.jsx -- read-only projection of Y.Text.length

```
const ytext = doc.getText('quill-content')
const update = () => setLength(ytext.length)
ytext.observe(update)    // re-run on every local or remote change
```

The "Realm Level" and XP bar are pure functions of `ytext.length` (`level = floor(length/200) + 1`). Because every peer's `Y.Text` converges to the same content (Section 6), this number is automatically identical on every peer's screen with no additional messages, no extra state in the document, and no coordination logic of its own -- it inherits the CRDT's consistency guarantee for free.

7.2 useToasts.js -- diffing the awareness-derived peer list

Join/leave toasts are computed entirely client-side by comparing the current `peers` array (Section 3.3) against a ref holding the previous render's set of client IDs, using a plain `Set` difference. The one-time "Party Assembled" achievement toast fires off a `useEffect` watching `status` transitioning to `'connected'`, guarded by a ref so it fires exactly once per mount. None of this reads or writes `Y.Doc` at all -- it is strictly a view over state the sync stack already produces.

Why this matters for the report:

The gamification adds zero new synchronization surface. There is nothing here that could desync between peers in a way the underlying CRDT/awareness system doesn't already handle -- a useful point to make if asked whether the theming complicated the distributed-systems core.

8. React-Specific Implementation Notes

8.1 No React.StrictMode

`main.jsx` deliberately does not wrap the app in `<React.StrictMode>`. In development, `StrictMode` intentionally double-invokes effects (mount, cleanup, mount again) to surface impure effects. That pattern is incompatible with a one-shot resource like a `WebRTCProvider/Y.Doc`: the lazy `useState` initializer only runs once, so the simulated cleanup-then-remount would call `provider.destroy()` on the one and only provider instance and then try to keep using it -- tearing down a freshly opened WebRTC session immediately after opening it. This is a known interaction across the Yjs/y-webrtc ecosystem generally, not specific to this app's code.

8.2 Editor.jsx avoids react-quill

Quill is instantiated manually inside a `useEffect`, against a plain `div` ref, instead of using the `react-quill` wrapper package. `react-quill` is unmaintained and calls the deprecated `ReactDOM.findDOMNode`, which throws/warns under React 18. Constructing Quill directly mirrors y-quill's own vanilla-JS usage examples and sidesteps that dependency entirely.

8.3 Room changes remount instead of mutating

```
// App.jsx
return <RoomApp key={roomId} roomId={roomId} identity={identity} />
```

Passing `roomId` as the React key means a different room produces a brand-new component instance -- and therefore a brand-new `Y.Doc/WebRTCProvider/IndexeddbPersistence` triple -- rather than trying to redirect a live provider to a different room name, which y-webrtc's API isn't designed to do safely.

9. Data Flow and Trust Boundary Summary

Document content (Y.Text updates)	Never touches the signaling server. Travels only over RTCDataChannel, directly between browsers, after the handshake completes.
Presence/cursor (awareness)	Same as above -- broadcast over the same data channel, never through the signaling server.
SDP offers/answers, ICE candidates	The only data that ever passes through the signaling server. Structurally cannot contain document text -- these are WebRTC/ICE protocol structures generated by the browser's own RTCPeerConnection implementation, not application data the app chooses to send.
Local document cache	Written to IndexedDB, scoped per browser, per room name. Never leaves the browser; not visible to the signaling server or any other peer directly (only via the normal sync process once reconnected).
Identity (name/avatar/color/class)	Generated client-side by <code>randomIdentity()</code> , cached in <code>sessionStorage</code> per tab. No server assigns or validates identities.

This table is effectively the argument for why the signaling server does not violate the project's "no central document authority" requirement: every row involving actual document or presence data is marked as never touching the server, and the one row that does pass through it is, by construction of the WebRTC/ICE protocols themselves, incapable of carrying document content.

See [SyncQuest_Project_Report.pdf](#) for the higher-level problem statement, use cases, and testing summary, and [README.md](#) / [ARCHITECTURE.md](#) in the project folder for setup instructions and the original diagrams.